Database & Infrastructure Troubleshooting Guide

SYSTEMS ENGINEERING & OPERATIONS MANUAL

1. Out of Memory (OOM) Errors

Quick Summary: An Out of Memory (OOM) error occurs when an application, database, or system server requests more operational RAM than is currently available or allocatable within the operating system pools.

Troubleshooting Methods

- **Identify the precise source:** Determine if the exhaustion is an `Application OOM` (heap limits exceeded), a `SQL Server OOM` (internal pool starvation), or an `Operating System OOM` (complete system RAM depletion).
- **Check active memory usage:**
  - *Windows Systems:* Leverage Task Manager or Resource Monitor to track process allocations.
  - *Linux Systems:* Execute commands like `top`, `htop`, or `free -h`.
  - *SQL Server:* Query internal memory Dynamic Management Views (DMVs).
- **Review recent systemic changes:** Assess if the event correlates with a new code deployment, a modified query launch, structural changes to ETL/Data loads, or sudden organic user traffic spikes.
- **Analyze SQL Server memory pools:** Check operational metrics through `sys.dm_os_process_memory` and verify that the `max server memory` configuration is appropriately defined to prevent host OS starvation.
- **Isolate memory-intensive queries:** Review active execution memory grants to pinpoint queries executing massive sort operations, hash joins, or large-scale data aggregations without proper indexing.
- **Examine long-running or blocking sessions:** Execute `sp_who2` or query `sys.dm_exec_requests` to identify non-yielding processes.

Immediate Mitigation Strategies

- Terminate or kill runaway queries consuming unsustainable memory grants.
- Stop or pause non-critical, problematic ETL jobs or batch processes immediately.
- If operating in a dynamic virtualized or cloud environment, provision additional RAM on-the-fly.
- Cycle or restart the target database service or host server only if completely locked or non-responsive.

Common Root Causes & Prevention

Root Causes: Highly inefficient SQL queries lacking index coverage; active memory leaks in application codebases; misconfigured or un-capped SQL Server memory allocation parameters; large concurrent data processing loads; sudden workload spikes.

Prevention: Properly configure explicit SQL Server memory limits; establish strict baseline utilization alerts; continuously optimize queries and maintain indexes; schedule heavy batch operations strictly during off-peak hours.

2. Network Interruption & Connectivity Errors

> **Quick Summary:** A network error indicates that the communication layer between the application, source systems, intermediate middleware, or the core database instances has been interrupted, dropped, delayed, or blocked entirely.

Troubleshooting Methods

- **Verify end-to-end connectivity:** Execute a basic `ping` to the destination server and ensure that the host machine is actively reachable over the network fabric.
- **Check DNS resolution paths:** Verify that the target hostname resolves to the correct IP address. Test direct connection via the raw IP address to isolate DNS mapping failures.
- **Validate active network ports:** Confirm that the database listening ports (e.g., SQL Server default port `1433`) are open and accessible. Inspect localized firewall rules and security groups using tools like `telnet` or PowerShell's `Test-NetConnection`.
- **Confirm server availability:** Validate that the source/target server is powered online and that the specific database instance service is operational.
- **Review distributed logs:** Inspect application error logs, database engine logs, and any available centralized network monitoring metrics.
- **Analyze network latency & packet loss:** Execute `tracert` (Windows) or `traceroute` (Linux) to map network hops, and check enterprise dashboards for packet drop metrics.

Immediate Mitigation Strategies

- Restart failed local network services or interfaces if safe to do so.
- Cycle or re-establish dropped corporate VPN tunnels or site-to-site connections.
- Reroute traffic through a verified backup network path or secondary gateway.
- Initiate a transaction retry block for transient errors or re-queue the failed job.
- Escalate directly to the network infrastructure and security teams for deep packet inspection.

Common Error Formats

- `A network-related or instance-specific error occurred while establishing a connection to SQL Server.`
- `Connection timed out / Login timeout expired.`
- `Unable to connect to the remote server / Host not found.`

"For network errors, verify server reachability, DNS resolution, port accessibility, server availability, and network latency. Identify whether the issue originates from the source system, target system, network infrastructure, or firewall configuration, then apply corrective actions accordingly.

***Note:** Technical teams often tend to blame the database tier first. Statistically, a surprising percentage of 'database issues' turn out to be a physical cable failure, an undocumented firewall rule change, a corrupted DNS cache, or a remote server that quietly stopped answering calls from the rest of civilization."*

3. Out of Disk Space Errors

Quick Summary: An Out of Disk Space error occurs when the database engine, local application logs, or host operating system cannot write data because the underlying physical storage volume has reached 100% capacity.

Troubleshooting Methods

1. **Audit available disk allocation:**
   - *Windows:* Use the `dir` command or review Disk Management consoles.
   - *Linux:* Run `df -h` to inspect mounted volumes.
2. **Identify space consumers:** Locate files driving the inflation. Primary database culprits include primary data files (`.mdf`), transaction log files (`.ldf`), the temporary database storage (`TempDB`), unpurged historic backups, or run-away application trace logs.
3. **Check database file growth properties:** In SQL Server, execute:

   ```
   EXEC sp_helpdb;
   ```

   Review the exact data/log sizes along with their configured autogrowth increments.
4. **Analyze transaction log consumption:** Inspect transaction log saturation by running:

   ```
   DBCC SQLPERF(LOGSPACE);
   ```

   Look for logs nearing full capacity, uncommitted long-running transactions, or broken/missing log backup schedules.
5. **Verify TempDB utilization:** Monitor for rapid TempDB growth caused by massive unindexed sort operations, heavy index rebuilds, or large-scale ETL temporary table caching.

Immediate Mitigation Strategies

- Purge or delete unnecessary files, obsolete trace logs, and compressed temp files.
- Safely move historical database backup files off the active drive to a separate network share or cold storage.

- Perform an intentional shrink of the transaction log file using `DBCC SHRINKFILE` (only when safe and the underlying root cause is addressed).
- Dynamically extend the underlying virtual machine storage volume or attach new physical disks.
- Temporarily stop or kill large data-ingest jobs to halt storage bleeding.

Common Error Formats

- `The disk is full.`
- `Could not allocate space for object in database because the filegroup is full.`
- `Operating system error 112: There is not enough space on the disk.`

4. SQL Server Connection Failures

> **Quick Summary:** This issue occurs when an external application client, automated ETL pipeline, or user session fails to successfully establish an authenticated connection handshake with the target SQL Server database instance.

Troubleshooting Methods & Verification Steps

- **Verify SQL Server service status:** Ensure the service is up. Check Windows Services for `SQL Server (MSSQLSERVER)` or named instances like `SQL Server (InstanceName)`.
- **Validate instance naming conventions:** Ensure connection strings match the exact naming format (e.g., `SERVER01`, `SERVER01\PROD`, or `SERVER01\DEV`). Typographical errors in instance paths are highly frequent.
- **Verify port accessibility:** Test connectivity to port `1433` via terminal tools:
  - *Command Prompt:* `telnet SERVER01 1433`
  - *PowerShell:* `Test-NetConnection SERVER01 -Port 1433`
- **Audit firewall parameters:** Validate that inbound network access rules permit communication through the primary database port and the SQL Browser service port (UDP `1434`) if named instances are utilized.
- **Review authentication modes:** Verify credential validity. Ensure the client is using the correct mechanism: Windows Authentication token vs. SQL Server Native Authentication (Username/Password), and verify that accounts are not locked out.
- **Examine the SQL Server Error Log:** Look explicitly for entries specifying `Login failed for user`, `Connection timeout expired`, or `Named Pipes Provider: Could not open a connection`.

Immediate Mitigation Strategies

- Start or cycle the stopped SQL Server service instance.
- Correct wrong host names, instance paths, or ports inside the active application connection string.
- Open necessary ports in local security firewalls or cloud network security groups (NSGs).
- Reset or update locked or expired service account credentials.
- Failover the environment to a secondary high-availability replica node if the primary host is corrupted.

Systemic Prevention Plan

- Deploy continuous up/down monitoring alerts for core SQL Server daemon processes.
- Maintain a central configuration repository tracking authoritative hostnames, named instances, and custom port mappings.
- Schedule automated log backups to keep disk space predictable and optimize database indexes weekly.
- Implement automated infrastructure connection health checks to proactively flag network degradation.